

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE CODE: CSA1401**

**COURSE NAME: COMPILER DESIGN**

### COURSE OUTCOME COVERED

**CO2:** Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

**Duration :60 Mins**

**On Class**

**QUESTIONS: 25**

**Total Marks:50**

Annexure A

### SHORT ANSWER TEST

#### *Code of Conduct:*

*I Dinesh V(192524309)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: Dinesh V***

Annexure A

---

### SHORT ANSWER TEST

Q1. A lexical analyzer produces the token stream: id

+ id \* id

Explain how the parser validates the syntactic structure of the expression.

Ans:

---

The parser receives the stream of tokens (id, +, id, \*, id) and matches them against the rules of a context-free grammar using top-down or bottom-up parsing. It checks whether the sequence forms a valid syntax tree and reports an error if the token order violates grammar rules

---

Q2. Given the grammar:

$$S \rightarrow aSb \mid \epsilon$$

Determine whether the string aaabbb belongs to the language using derivation.

Ans:

$$\begin{aligned} S &\Rightarrow aSb \\ &\Rightarrow aaSbb \\ &\Rightarrow aaaSbbb \\ &\Rightarrow aaa\epsilon bbb = aaabbb \end{aligned}$$

**Conclusion:** The string **aaabbb belongs to the language** generated by the grammar.

Q3. Remove left recursion from the grammar: E

$$\rightarrow E + T \mid T$$

Ans:

Applying the rule  $A \rightarrow A\alpha \mid \beta \Rightarrow A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$ :

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \end{aligned}$$

Q4. Find FIRST(A) for the grammar:

$$A \rightarrow aB \mid \epsilon B$$

$$\rightarrow b$$

Ans:

$$\text{FIRST}(A) = \{a, \epsilon\}$$

(Note: If  $\epsilon$  represents

$\epsilon$ ,

$$\text{FIRST}(A) =$$

$$\{a, \epsilon\}.$$

Q5. Determine whether the grammar is suitable for top-down parsing:

$$A \rightarrow Aa \mid b$$

**Ans: No.** The grammar contains **immediate left recursion** ( $A \rightarrow Aa$ ), which causes top-down parsers (like predictive or recursive descent parsers) to enter an infinite loop.

Q6. Apply left factoring to the grammar: S

$$\rightarrow \text{while } E \text{ do } S \mid \text{while } E \text{ do } S \text{ else } S$$

Ans:

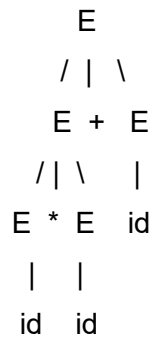
$$S \rightarrow \text{while } E \text{ do } S S' \quad S' \rightarrow \text{else } S \mid \epsilon$$

Q7. Construct a parse tree for the expression:

id \* id + id using the

grammar:

$E \rightarrow E + E \mid E * E \mid id$



Q8. Write the recursive descent procedures for the grammar:

$S \rightarrow aA$

$A \rightarrow \quad \rightarrow \quad bA \quad \mid \quad c$

Ans:

```

void S() {
    if (lookahead == 'a') { match('a'); A(); }
    else error();
}

void A() {
    if (lookahead == 'b') { match('b'); A(); }
    else if (lookahead == 'c') { match('c'); }
    else error();
}
  
```

Q9. Given the grammar:

$S \rightarrow AB$

$A \rightarrow a \mid \epsilon$

$B \rightarrow b$

Construct the predictive parsing table.

$FIRST(S) = \{a, e\}$ ,  $FIRST(A) = \{a, e\}$ ,  $FIRST(B) = \{b\}$ .

| Non-Terminal | a                  | b | e                  | \$ |
|--------------|--------------------|---|--------------------|----|
| S            | $S \rightarrow AB$ |   | $S \rightarrow AB$ |    |

| Non-Terminal | a                 | b                 | e                 | \$ |
|--------------|-------------------|-------------------|-------------------|----|
| $A$          | $A \rightarrow a$ |                   | $A \rightarrow e$ |    |
| $B$          |                   | $B \rightarrow b$ |                   |    |

Q10. Demonstrate the first three actions of a shift-reduce parser for: id

+

id

**Ans:**

1. **Shift** id onto stack.
2. **Reduce** id using production  $E \rightarrow \text{id}$ .
3. **Shift** + onto stack.

Q11. Identify the handle in the expression:

id \* id + id for the

grammar:

$E \rightarrow E + E \mid E * E \mid \text{id}$

**Ans:**

The first handle recognized and reduced is the leftmost terminal **id** (reduced via  $E \rightarrow \text{id}$ ). Subsequently, for operator precedence evaluation, the handle is id \* id (reduced via  $E \rightarrow E * E$ ).

Q12. Construct the operator precedence relations for the operators:

+, \* where \* has higher precedence

than +.

**Ans:**

- $+\cdot*$  (+ yields precedence to \*)
- $*\cdot>+$  (\* takes precedence over +)
- $+\cdot>+$  (Left associativity of +)
- $*\cdot>*$  (Left associativity of \*)

Q13. Explain how operator precedence parsing handles the expression:

$\text{id} + \text{id} * \text{id}$

Ans:

It evaluates precedence relations between adjacent operators: since  $+ < \cdot$ , the parser shifts past  $+$  and  $\text{id}_2$ , then reduces  $\text{id}_2 * \text{id}_3$  first due to  $\cdot > \$$ , ensuring multiplication takes precedence over addition.

Q14. What are LR parsers? Illustrate with a simple parsing scenario.

Ans:

LR parsers are deterministic bottom-up parsers that scan input **Left-to-right** and construct a **Rightmost** derivation in reverse.

*Scenario:* Given stack  $\$ 0$  and input  $\text{id } \$$ , the parser looks up the action table at state 0, sees  $s_2$  (shift state 2), moves  $\text{id}$  to the stack, and transitions to state  $\$ 0 \text{ id } 2$ .

Q15. Given the grammar:

$S \rightarrow AA$

$A \rightarrow aA \mid b$

Write the steps involved in constructing an SLR parsing table.

Ans:

1. Construct the augmented grammar  $S' \rightarrow S$ .
2. Compute canonical collection of LR(0) item sets and transitions.
3. Calculate FOLLOW sets for all non-terminals ( $\text{FOLLOW}(S) = \{\$ \}$ ,  $\text{FOLLOW}(A) = \{a, b, \$\}$ ).
4. Fill the ACTION table: shift for transitions on terminals, reduce  $A \rightarrow \alpha$  on terminals  $a \in \text{FOLLOW}(A)$ .
5. Fill the GOTO table for non-terminal state transitions.

Q16. Compute FOLLOW(A) for the grammar:

$S \rightarrow AB$

$A \rightarrow a \mid \varepsilon B$

$\rightarrow \quad b$

**Answer:**

- For  
 $S \rightarrow$   
 $A$  ,  
 $\text{FOLLOW}(S) \subseteq$   
 $\text{FOLLOW}(A) \Rightarrow$   
 $\$ \in$   
 $\text{FOLLOW}(A)$   
 .
- For  
 $S \rightarrow$   
 $AB$  ,  
 $\text{FIRST}(B) \subseteq$   
 $\text{FOLLOW}(A) \Rightarrow$   
 $b \in$   
 $\text{FOLLOW}(A)$   
 .

$$\text{FOLLOW}(A) = \{b, \$\}$$

Q17.

Construct

the

augmented

grammar

for:  $S \rightarrow$

$aA$

$A \rightarrow b$

Q18. Find the closure of the LR(0) item:

$S' \rightarrow \bullet S$  for the grammar:

$S \rightarrow aAA$

$\rightarrow b$

$$\text{Closure}(S' \rightarrow \bullet S) = \{S' \rightarrow \bullet S, S \rightarrow \bullet aA\}$$

Q19. Check whether the grammar is ambiguous:

$E \rightarrow E + E \mid \text{id}$  using the string:  $\text{id} + \text{id} + \text{id}$

**Ans:**

**Yes, it is ambiguous.** The string yields two distinct leftmost derivations / parse trees:

1.  $(E + E) + E \Rightarrow$  Left-associative structure

2.  $E + (E + E) \Rightarrow$  Right-associative structure

Q20. Construct a leftmost derivation for:

$\text{id} + \text{id} * \text{id}$  using:

$E \rightarrow E + E \mid E * E \mid \text{id}$

**Ans:**

$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow \text{id} + E \\ &\Rightarrow \text{id} + E * E \\ &\Rightarrow \text{id} + \text{id} * E \\ &\Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$

Q21. Apply left factoring to:  $S$

$\rightarrow aBc \mid aBd \mid b$

**Ans:**

$$S \rightarrow aBS' \mid b$$

$$S' \rightarrow c \mid d$$

Q22. Find  $\text{FIRST}(F)$  for:  $F$

$\rightarrow (E) \mid \text{id}$

**Ans:**

$$\text{FIRST}(F) = \{(, \text{id}\}$$

Q23. Determine whether the grammar is suitable for predictive parsing:

$S \rightarrow aA \mid aB$

$A \rightarrow b$

$B \rightarrow c$

Justify your answer.

**Ans: No.** The productions  $S \rightarrow aA$  and  $S \rightarrow aB$  both begin with the same terminal  $a$ . This causes a **FIRST/FIRST conflict** in the predictive parsing table at entry  $M[S, a]$ . Left factoring is required first.

Q24. Construct recursive descent parser procedures for:  $S$

$\rightarrow abS \mid \epsilon$

Ans:

```
void S() {
    if (lookahead == 'a') {
        match('a');
        match('b');
        S();
    } else {
        // epsilon production: do nothing
        return;
    }
}
```

Q25. Build the predictive parsing table for:  $S \rightarrow aS \mid b$  and show the table entries for terminals  $a$ ,  $b$ , and  $\$$ .

Ans:

**Answer:**

$\text{FIRST}(aS) = \{a\}$ ,  $\text{FIRST}(b) = \{b\}$ .

| Non-Terminal | a                 | b                 | \$ |
|--------------|-------------------|-------------------|----|
| $S$          | $S \rightarrow a$ | $S \rightarrow b$ |    |

**RUBRICS FOR CALCULATION**

| <b>Criteria</b>           | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                  | <b>Level 3 – Proficient</b>             | <b>Level 2 – Developing</b>                  | <b>Level 1 – Beginning</b>               |
|---------------------------|------------------|-------------------------------------------------------------|-----------------------------------------|----------------------------------------------|------------------------------------------|
| Understanding of Concepts | 25%              | Demonstrates thorough, accurate understanding of concepts   | Good understanding with minor gaps      | Basic understanding with some misconceptions | Poor understanding; major misconceptions |
| Explanation               | 25%              | Detailed, well explained answers with relevant examples     | Clear explanation with adequate details | Limited explanation; lacks depth             | Poor or unclear explanation              |
| Application               | 20%              | Effectively applies concepts with strong analytical ability | Applies concepts with minor errors      | Limited application with weak analysis       | No application or incorrect analysis     |
| Organization              | 15%              | Well-structured answers with logical flow and coherence     | Organized with minor issues in flow     | Basic structure, lacks clarity               | Poor organization, incoherent answers    |
| Accuracy                  | 10%              | Completely accurate and covers all required points          | Mostly accurate with minor omissions    | Partially correct with missing points        | Incorrect answers with major gaps        |
| Clarity                   | 5%               | Clear, neat, professional presentation                      | Understandable with minor issues        | Limited clarity, readability issues          | Poorly written, difficult to understand  |